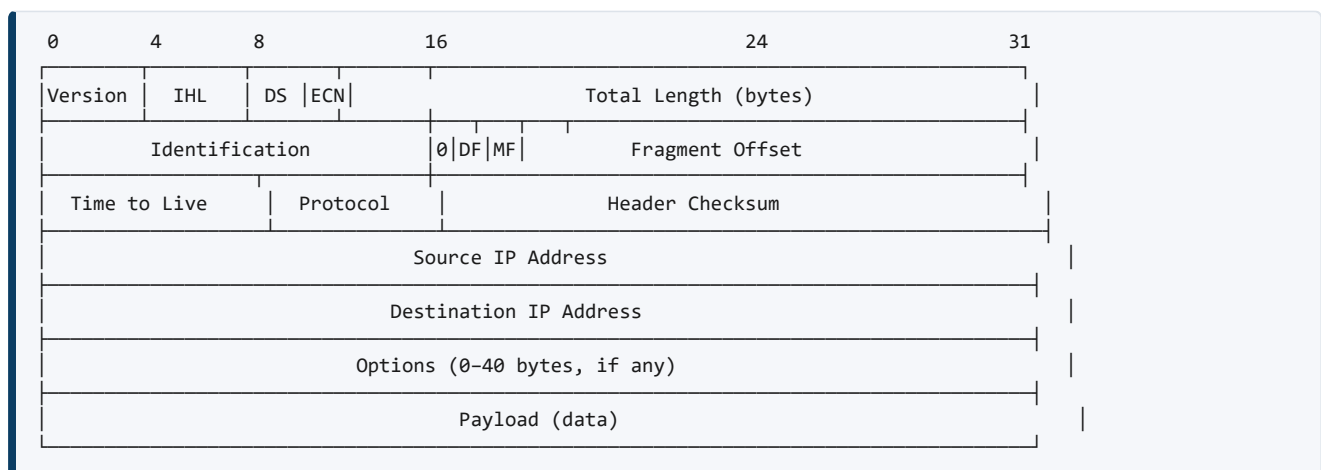


IPv4 Header, Fragmentation, DHCP & NAT

PART 1 — IPv4 Header

1. Header Layout



- **Header size:** 20 bytes (no options) to 60 bytes (max options).
- **Total length:** 20 bytes to 65,535 bytes (a 16-bit field).

2. Field-by-Field

- **Version (4 bits)** — currently 4 (IPv4).
- **IHL / Header Length (4 bits)** — header length **in units of 4-byte words**. Value ranges 5 ($5 \times 4 = 20$, minimum, no options) to 15 ($15 \times 4 = 60$, maximum, full options).
- **DS/ECN (1 byte)** — historically "Type of Service."
- **DS (Differentiated Services, 6 bits)** — lets traffic be prioritized (e.g., mark premium/real-time traffic for preferential handling).
- **ECN (Explicit Congestion Notification, 2 bits)** — lets routers mark a packet to signal *incipient* congestion, so the transport layer (TCP) can react *before* an actual packet drop occurs.
- **Total Length (16 bits)** — header + payload, in bytes.
- **Identification (16 bits)** — a unique number (per source host) assigned to a datagram; **all fragments of the same original datagram share this same value**, which is exactly how the receiver knows which fragments belong together during reassembly.
- **Flags (3 bits):**
 - Bit 0: reserved, always 0.

- **DF (Don't Fragment)** — if set, this datagram must **never** be fragmented; if it's too large for a link's MTU, it's simply **dropped**, and (per ICMP) an error is typically reported back to the sender instead.
- **MF (More Fragments)** — set on every fragment **except the last one** of a fragmented datagram; the last fragment has **MF = 0**, signaling reassembly is complete once it arrives.
- **Fragment Offset (13 bits)** — where in the *original* datagram this fragment's data begins, measured in **units of 8 bytes** (not 1 byte) — this is why fragment sizes (except the last fragment) must always be a multiple of 8 bytes.
- **TTL (Time To Live, 1 byte)** — the maximum number of routers (hops) the packet may pass through. **Every router decrements it by 1**; when it hits 0, the packet is dropped (and an ICMP "Time Exceeded" is typically sent back). **Purpose:** guarantees a packet caught in a routing loop is eventually discarded, rather than circulating forever.
- **Protocol (1 byte)** — identifies the higher-layer protocol carried in the payload, so the receiving host knows which layer to hand the data to:

Value	Protocol
1	ICMP
2	IGMP
4	IP-in-IP encapsulation
6	TCP
17	UDP

- **Header Checksum (16 bits)** — a 1's complement sum over **only the header** (not the payload — that's checked separately by the transport layer's own checksum). **Recomputed at every hop**, because TTL (which is inside the header) changes at every hop.
- **Source / Destination IP Address** (32 bits each).
- **Options (0–40 bytes)** — optional features, e.g.:
- **Record Route** — each router that handles the packet appends its own IP address to the header, building a hop-by-hop path trace.
- **Timestamp** — each router appends its IP and the time it processed the packet, useful for measuring per-hop delay/latency.
- **Source Routing** — the sender specifies a mandatory list of routers the packet must pass through.
- **Padding** — extra bytes to ensure the header ends on a 4-byte boundary (since IHL counts in 4-byte units).

Numerical practice — header length

Q: HLEN = 5, Total Length = **(0028)₁₆** . How much payload data is being carried?

Header length = $5 \times 4 = 20$ bytes
Total length = $0x0028 = 40$ bytes
Payload = $40 - 20 = 20$ bytes

Q: HLEN = 1000 in binary. How many bytes of options are present?

HLEN (binary 1000) = 8 (decimal)
Header length = $8 \times 4 = 32$ bytes
Options = $32 - 20$ (minimum header size) = 12 bytes

3. MTU and Fragmentation

MTU (Maximum Transmission Unit) — the largest datagram size a given data-link protocol can carry in one frame. IP's own maximum is 65,535 bytes, but the *data link* layer imposes a much smaller practical limit (Ethernet: 1500 bytes; PPP: 296 bytes).

When a router needs to forward a datagram onto a link whose MTU is smaller than the datagram's size, it must fragment it.

Worked example

A host sends a datagram with **4000 bytes of data** (20-byte header, so total = 4020 bytes). It must cross a link whose MTU is **1500 bytes**.

Max data per fragment = $\text{floor}((\text{MTU} - \text{header size}) / 8) \times 8$
= $\text{floor}((1500 - 20) / 8) \times 8$
= $\text{floor}(1480 / 8) \times 8$
= $185 \times 8 = 1480$ bytes (conveniently already a multiple of 8 here)

Number of fragments needed = $\text{ceil}(4000 / 1480) = 3$

Fragment	Data size	Offset (in 8-byte units)	Total length (data+header)	MF
1	1480 bytes	0	1500	1
2	1480 bytes	$1480 / 8 = 185$	1500	1
3	$4000 - 1480 - 1480 = 1040$ bytes	$(1480 + 1480) / 8 = 370$	1060	0

- All three fragments carry the **same Identification value** as the original datagram.
- Each fragment gets its **own full 20-byte IP header** (so it can be routed completely independently).
- **A fragment can itself be fragmented again**, if it later crosses an even smaller-MTU link — fragmentation offset arithmetic composes correctly because offset is always measured relative to the *original* datagram, not the immediately preceding fragment.

Reassembly, at the destination

- Fragments can **arrive out of order** — the destination uses the offset field to reconstruct the correct sequence, regardless of arrival order.
- The destination doesn't know how much buffer space to reserve until the **final fragment** (**MF=0**) arrives, since that's what reveals the complete original size.
- Duplicate fragments may arrive — the destination keeps only one copy.
- If some fragment **never arrives**, the destination eventually **times out** and discards the entire partially-reassembled datagram (there's no way to "patch a gap" — the datagram is simply lost as a whole).

Interview soundbite: "Fragmentation is invisible to the transport layer — TCP/UDP see one segment go out and (ideally) one segment come back together at the far end. It's purely an IP-layer mechanism to work around whatever the smallest MTU happens to be along the path, and the offset field's 8-byte granularity is exactly why fragment sizes have to be multiples of 8 (except the very last fragment, which can be any size)."

DF flag interaction: if DF is set and a router encounters a link whose MTU is too small, the packet is simply **dropped** rather than fragmented — this is precisely the mechanism **Path MTU Discovery** uses: send probes with DF set, and use the resulting ICMP errors to discover the smallest MTU along an entire path, all without ever needing to actually fragment anything.

PART 2 — DHCP (Dynamic Host Configuration Protocol)

4. What DHCP Automates

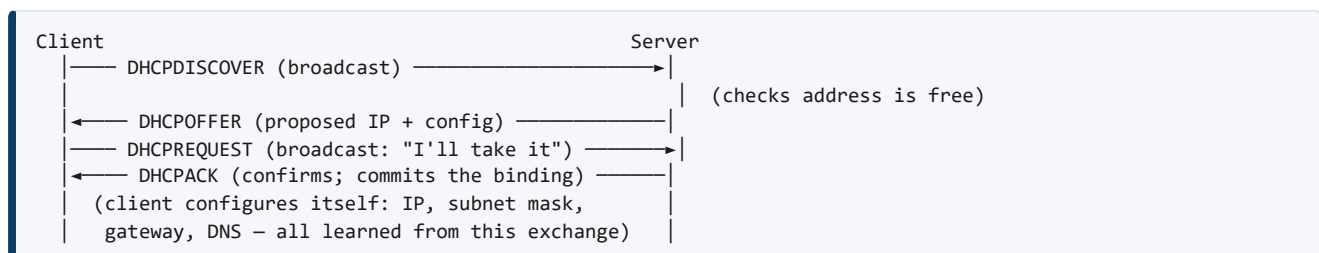
Without DHCP, a network administrator would have to manually configure an IP address, subnet mask, default gateway, and DNS server on **every single device** — DHCP automates all of this, dynamically assigning configuration on demand.

- DHCP is an extension of the older **BOOTP** (Bootstrap Protocol), which required manual pre-configuration in a server database; DHCP adds dynamic, lease-based allocation on top.
- **Components:** DHCP Server, DHCP Client, and an optional DHCP/BOOTP Relay Agent (needed when the client and server are on different subnets, since DHCP's initial discovery is a broadcast, which routers don't normally forward).

5. DHCP Message Types

Message	Direction	Purpose
DHCPDISCOVER	Client → (broadcast)	"Is any DHCP server out there?"
DHCPOFFER	Server → Client	"Here's a proposed IP + config"
DHCPREQUEST	Client → (broadcast)	Accept one specific offer (and implicitly decline all others); or renew/confirm an existing lease
DHCPACK	Server → Client	Confirms and commits the assignment
DHCPNAK	Server → Client	Rejects the client's request (e.g., address no longer valid, client moved subnets)
DHCPDECLINE	Client → Server	"That address is already in use by someone else" (client detected a conflict, e.g., via gratuitous ARP)
DHCPRELEASE	Client → Server	Client voluntarily gives up its address before the lease expires
DHCPINFORM	Client → Server	"I already have an IP — just send me other config info (DNS, etc.)"

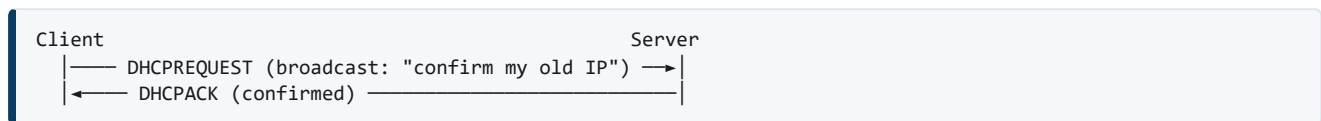
6. Allocating a New Address — the DORA sequence



DORA = **D**iscover → **O**ffer → **R**equest → **A**cknowledge.

- **Not just an IP is assigned** — the OFFER/ACK also typically carries the subnet mask, default gateway, and DNS server addresses.
- If the server ultimately can't honor the request (e.g., another client took the address in the meantime), it sends **DHCPNAK** instead of ACK.
- If the client, after receiving the ACK, discovers via ARP that the address is already in use by another device, it sends **DHCPDECLINE**.

7. Reusing a Previously-Held Address (renewal)



The server generally **should not** re-check address availability in this flow, since the client already legitimately held this address.

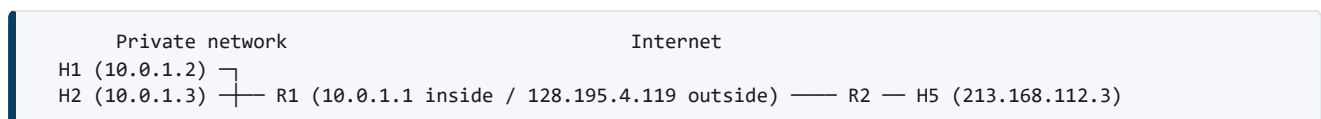
8. Lease Time

- The DHCP server defines how long a client may hold an address — the **lease time**.
- The client requests renewal at **50% of the lease time elapsed** — proactively, well before the lease actually expires, to avoid any gap in connectivity.
- If the lease fully expires without renewal, the client must restart the full DORA process to get a (possibly different) address.
- If a client no longer wants its address before the lease expires, it explicitly gives it up via **DHCPRELEASE**.

PART 3 — NAT (Network Address Translation)

9. Why NAT Exists

- A **short-term mitigation** for IPv4 address exhaustion (the long-term fix being IPv6; CIDR is another short-term mitigation, covered in the IP Addressing topic).
- **Core idea:** map multiple **private**, non-routable addresses (from the private ranges covered in the IP Addressing topic) to one (or a small pool of) **public**, routable address(es) — hiding an entire internal network behind a much smaller number of externally-visible addresses.
- **The router performs all translation** — hosts inside the private network are completely unaware NAT is even happening.
- **Only the source IP is translated on the way out** (and correspondingly the destination IP on the way back in) — the packet's actual payload/destination (for outbound traffic) is untouched.



10. Static vs Dynamic NAT

- **Static NAT** — a fixed, permanent **one-to-one** mapping between a specific private address and a specific public address. Used when a specific internal device (e.g., a server) needs a consistent, predictable public identity.
- **Dynamic NAT** — a public address is picked **from a pool** of available public addresses for each new session, rather than a fixed 1:1 mapping — still one private-to-one-public at any given moment, but *which* public address is used can vary.

11. PAT / NAPT (Port Address Translation, "Masquerading")

The problem with static/dynamic NAT: both still require roughly as many public addresses as simultaneously active private hosts — not much of an address-saving win if you have hundreds of internal hosts.

PAT (also called NAT overloading) solves this: it's a genuine **many-to-one** mapping — many private hosts share a **single** public IP address simultaneously, distinguished from each other by **TCP/UDP port numbers**.

Worked example:

```
10.4.4.1 : (its own local port) → 2.2.2.2 : TCP source port 1923
10.4.4.5 : (its own local port) → 2.2.2.2 : TCP source port 1924
```

Both internal hosts appear to the outside world as the **same** public IP (**2.2.2.2**), but the router keeps an internal table mapping (public IP, public port) ↔ (private IP, private port) for each active connection — when a reply comes back addressed to **2.2.2.2:1923**, the router knows (from its table) to translate it back to **10.4.4.1**, and similarly for **2.2.2.2:1924** → **10.4.4.5**.

- This is exactly what lets an entire home network (potentially dozens of devices) share just **one** public IP address from an ISP.

Interview Questions With Answers

Q1. Why is the IP header checksum recomputed at every router hop, rather than being computed once by the original sender and left unchanged?

Answer: The header checksum covers only the header fields, and one specific header field — TTL — is deliberately decremented by every router that forwards the packet. Since the header's contents literally change at every hop, the checksum (which is a function of those exact contents) must be recomputed at each hop to remain valid; if it were left as originally computed, it would immediately

and incorrectly flag every forwarded packet as "corrupted" the moment any router decremented the TTL.

Q2. Why must fragment sizes (except the last fragment) be multiples of 8 bytes?

Answer: Because the Fragment Offset field measures the fragment's position within the original datagram in units of **8 bytes**, not 1 byte — this compact encoding is what lets a 13-bit offset field represent positions across the full 65,535-byte maximum datagram size. If a fragment's data size (other than the very last one) weren't a multiple of 8, the next fragment's starting position couldn't be expressed exactly by this offset field, breaking correct reassembly.

Q3. What happens if the DF (Don't Fragment) flag is set and a packet needs to cross a link with too small an MTU?

Answer: The packet is dropped outright rather than fragmented — DF explicitly forbids fragmentation for that datagram. Typically, the router that had to drop it sends back an ICMP error to the original sender, informing it that fragmentation would have been required. This exact mechanism is deliberately exploited by Path MTU Discovery: sending probe packets with DF set and using the resulting ICMP errors (and the MTU value they can report) to determine the smallest MTU along an entire path without ever needing to actually fragment real data.

Q4. Why is a DHCP relay agent needed when the client and server are on different subnets?

Answer: The very first message in DHCP's DORA sequence, DHCPDISCOVER, is sent as a **broadcast**, since the client doesn't yet have any IP configuration (not even a gateway) and doesn't know where a DHCP server actually is. Routers, by design, don't forward broadcast traffic between subnets — so if the DHCP server sits on a different subnet than the client, the broadcast would never reach it. A relay agent, sitting on the client's own subnet, listens for these broadcasts and forwards them (as a normal unicast) to the actual DHCP server on another subnet, then relays the server's response back.

Q5. Why does the client request lease renewal at 50% of the lease time, rather than waiting until the lease is about to expire?

Answer: Renewing early, well before expiration, builds in a safety margin — if the renewal request happens to fail or the DHCP server is temporarily unreachable, the client still has roughly half its original lease time remaining to retry before the address actually becomes invalid. Waiting until the last moment to renew would risk the lease expiring (and the client losing its IP address, disrupting connectivity) if even a single renewal attempt failed or was delayed.

Q6. What's the fundamental difference between static NAT, dynamic NAT, and PAT?

Answer: Static NAT is a fixed, permanent one-to-one mapping between one specific private address and one specific public address. Dynamic NAT is also one-to-one at any given moment, but the specific public address assigned is picked from a shared pool rather than being fixed — different

private hosts might get a different public address on different occasions. PAT (Port Address Translation) is fundamentally different in kind, not just degree: it's a many-to-one mapping, where multiple private hosts share the exact same single public IP address *simultaneously*, distinguished from each other using TCP/UDP port numbers rather than needing a distinct public IP per host at all.

Q7. Why does PAT save far more public IP addresses than static or dynamic NAT?

Answer: Static and dynamic NAT both still require roughly one public address per simultaneously active private host (even if dynamic NAT's assignment isn't fixed, it's still consuming one pool address per active session). PAT breaks this 1-address-per-host requirement entirely by using port numbers to multiplex many private hosts' traffic onto a *single* shared public IP address — since a single IP address has 65,536 possible port numbers, PAT can support many more simultaneous private hosts sharing that one address than static/dynamic NAT could ever support with the same single address.

Q8. Scenario: A large file is fragmented into 3 IP fragments during transit. The second fragment is lost due to a transient network issue, but the first and third arrive successfully. What happens at the destination, and does IP itself retransmit the missing fragment?

Answer: The destination cannot reassemble the original datagram with a gap in the middle — it will hold the successfully-received fragments (1 and 3) in a reassembly buffer, waiting for the missing fragment (2) to arrive, until a reassembly timer expires. IP itself has **no retransmission mechanism** — it's a best-effort, connectionless protocol at this layer — so once the reassembly timer expires, the destination simply **discards the entire partially-reassembled datagram**, including the fragments that did arrive successfully. Recovery, if it happens at all, is the responsibility of a higher layer: if the original data was carried over TCP, TCP's own retransmission mechanism (having never received an acknowledgment for that segment) will eventually retransmit the entire original segment from scratch — IP fragmentation and reassembly are completely invisible to, and independent of, that higher-layer reliability mechanism.